

**ALL PROGRAMMABLE**

**ANY MEDIA**

**5G**

**4K/8K**

**ANY STANDARD**

**ANY MACHINE**

**ANY NETWORK**

5G Wireless • Embedded Vision • Industrial IoT • Cloud Computing



PYNQ™ Binarized Neural Network

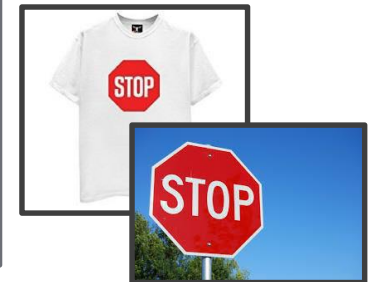
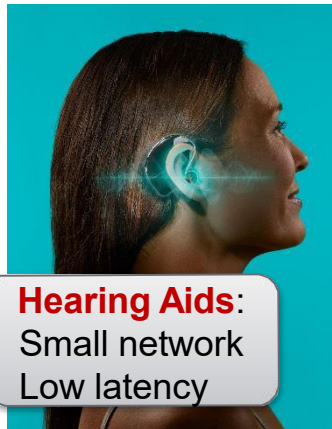
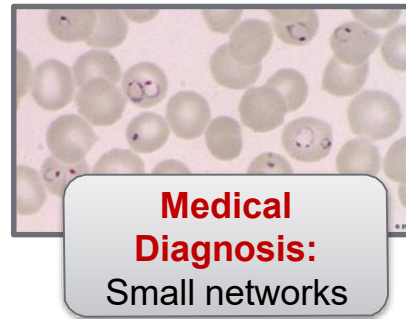
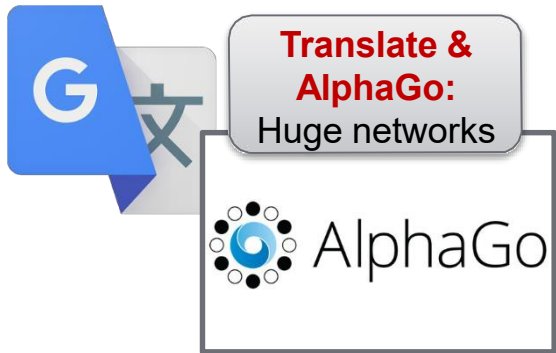


# Agenda

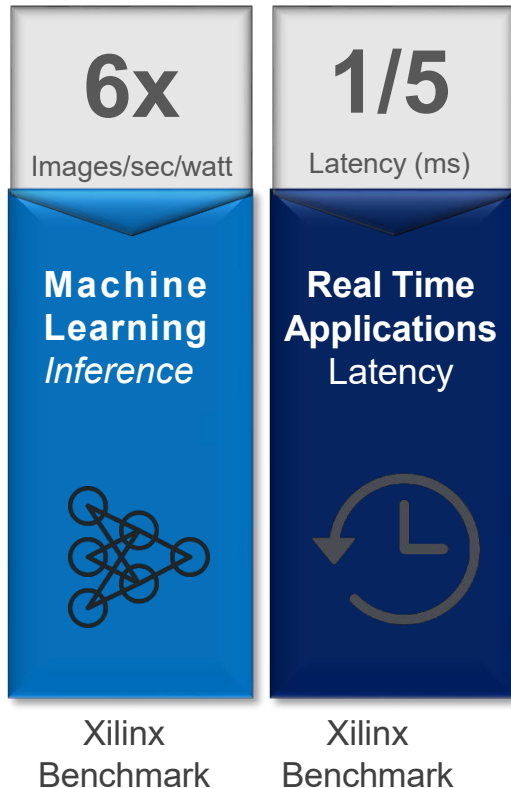
- Why use Zynq for DL Inference
- Labs:
  - MNIST
  - Char Recognition
  - Cifar 10
  - Road Signs

# Why use Zynq for DL Inference

# Diverse Applications with Diverse Design Targets



# Zynq SoCs Offer Superior Throughput, Latency



| GoogLeNet @ batch = 1 |               | Xilinx ZU9 | Xilinx ZU5 | eGPU* |
|-----------------------|---------------|------------|------------|-------|
|                       | Images/s      | 370.0      | 155.0      | 70    |
|                       | Power (W)     | 7.0        | 4.5        | 7.9   |
|                       | Images/s/watt | 53.0       | 34.5       | 8.9   |

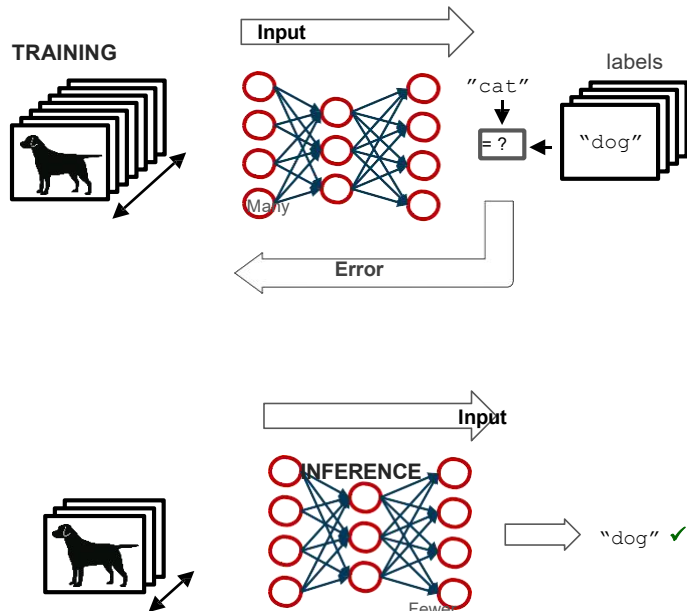
| GoogLeNet @ batch = 1 |              | Xilinx ZU9 | Xilinx ZU5 | eGPU* |
|-----------------------|--------------|------------|------------|-------|
|                       | Images/s     | 370.0      | 155.0      | 70    |
|                       | Latency (ms) | 2.7        | 6.4        | 14.2  |

| GoogLeNet @ batch = 8 |              | Xilinx ZU9 | Xilinx ZU5 | eGPU* |
|-----------------------|--------------|------------|------------|-------|
|                       | Images/s     | 370.0      | 155.0      | 163   |
|                       | Latency (ms) | 2.7        | 6.4        | 49.0  |

**For large batch,  
CPU/GPU/DSPs latency  
increases significantly**

\* eGPU = nVidia TX1: nVidia GoogLeNet performance: <https://devblogs.nvidia.com/paralleforall/jetpack-doubles-jetson-tx1-deep-learning-inference/>

# The Divergence of Training and Inference



**Training:** Process for machine to “learn” and optimize model from data

**Inference:** Using trained models to predict/estimate outcomes from new observations in efficient deployments

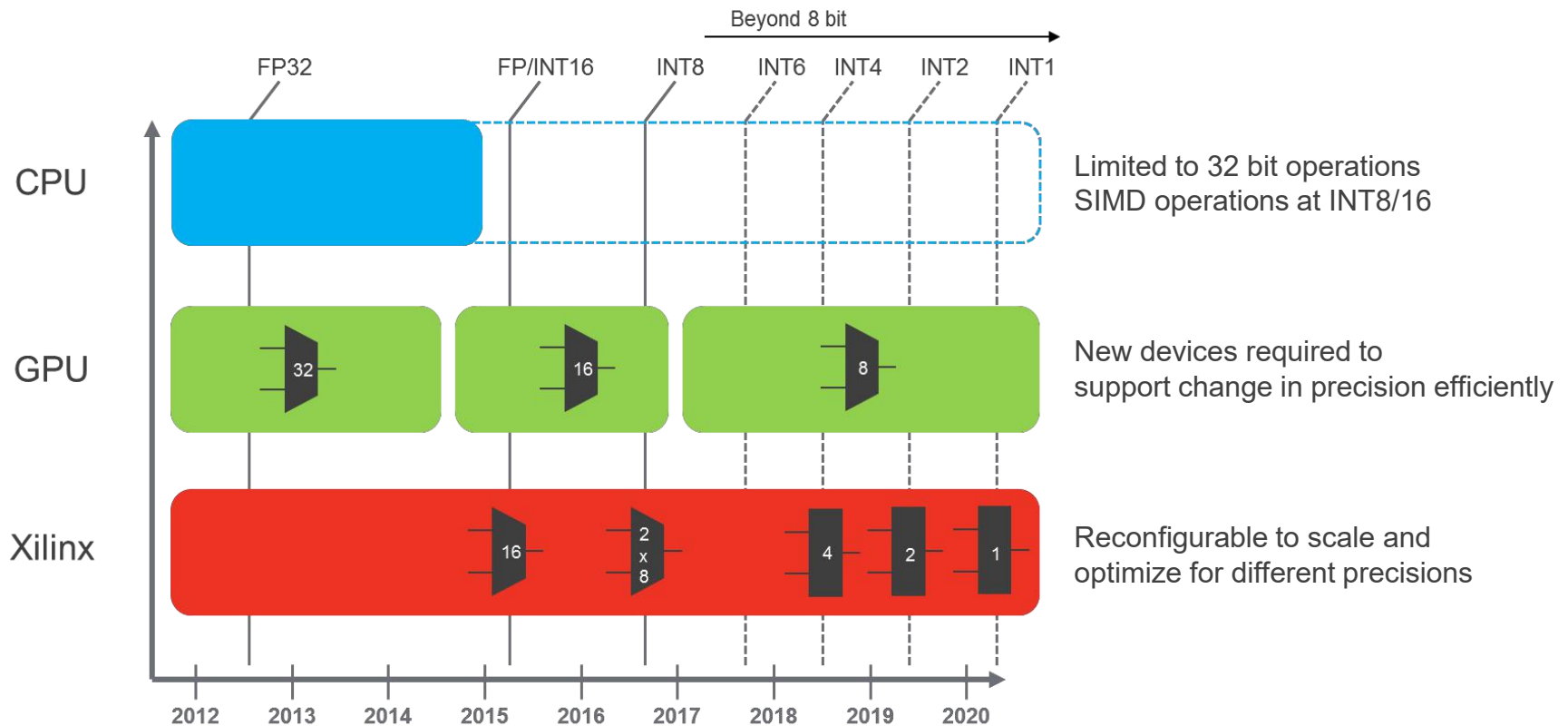
*Inference now 8 bit and below for maximum efficiency*



| Top-5 Accuracy | FP-32 | FIXED-16 (INT16) | FIXED-8 (INT8) | Difference vs FP32 |
|----------------|-------|------------------|----------------|--------------------|
| VGG-16         | 86.6% | 86.6%            | 86.4%          | (0.2%)             |
| GoogLeNet      | 88.6% | 88.5%            | 85.7%          | (2.9%)             |
| SqueezeNet     | 81.4% | 81.4%            | 80.3%          | (1.1%)             |

<https://arxiv.org/pdf/1510.00149v5.pdf>

# Future Proof Architecture for Any Precisions



# BNN

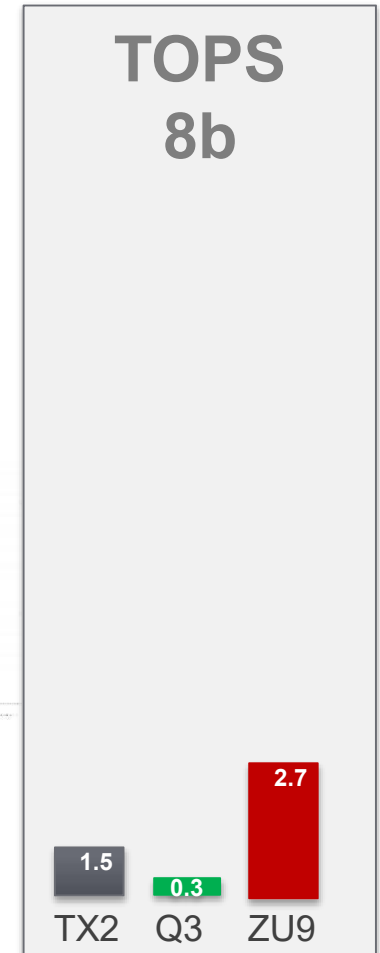


# BNN: Unparalleled Performance

- Reducing precision from 8b to 1b shrinks LUT cost by **40x**
- Potential to scale CNN performance to above **23TOPS (ZU9)**

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |

- Assuming 300 MHz with 90%/70% DSP/LUT utilizations
- Resource consumption assumption: 2.5 LUTs/op (INT1), 16 LUTs/op (INT4), 0.25 DSP/op (INT8)

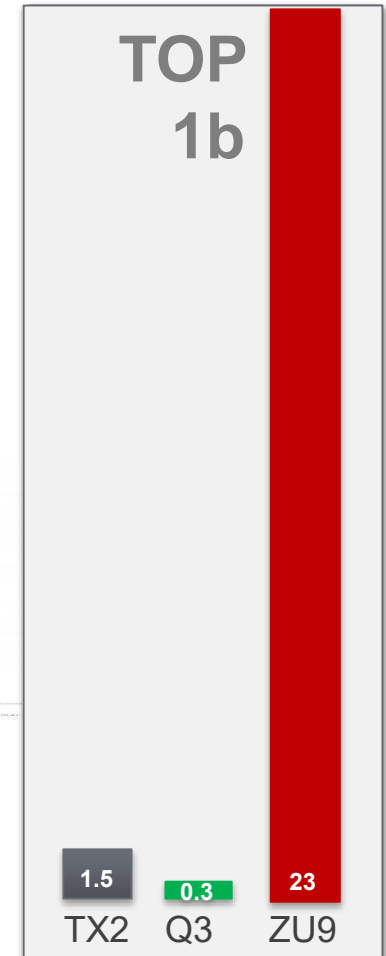


# BNN: Unparalleled Performance

- Reducing precision from 8b to 1b shrinks LUT cost by **40x**
- Potential to scale CNN performance to above **23TOPS (ZU9)**

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |

- Assuming 300 MHz with 90%/70% DSP/LUT utilizations
- Resource consumption assumption: 2.5 LUTs/op (INT1), 16 LUTs/op (INT4), 0.25 DSP/op (INT8)

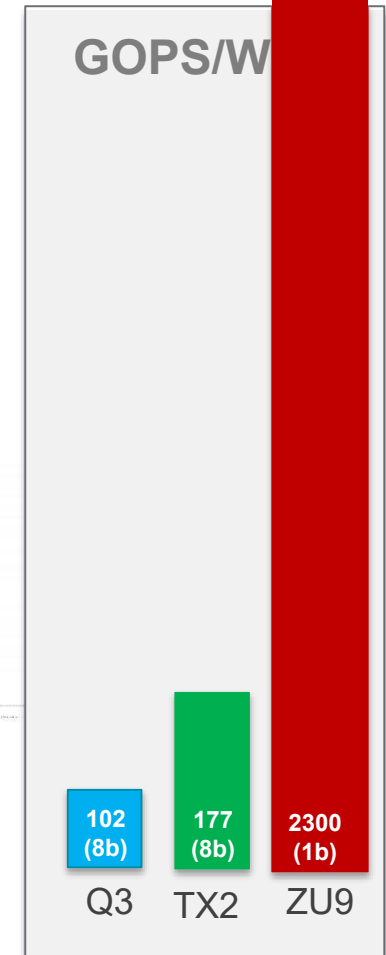


# BNN: Unparalleled Performance

- Reducing precision from 8b to 1b shrinks LUT cost by **40x**
- Potential to scale CNN performance to above **23TOPS (ZU9)**

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |
| 1b | 1b | 1b | 1b | 1b | 1b | 1b | 1b |

- Assuming 300 MHz with 90%/70% DSP/LUT utilizations
- Resource consumption assumption: 2.5 LUTs/op (INT1), 16 LUTs/op (INT4), 0.25 DSP/op (INT8)

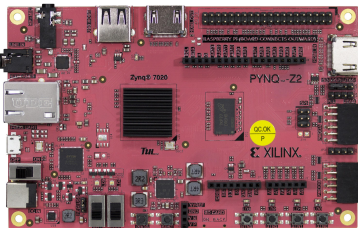


# PYNQ BNN overlays

|     | Z7020 ARM FPS<br>(software only) | Raspberry Pi3<br>FPS |
|-----|----------------------------------|----------------------|
| LFC | 17.3                             | 44.3                 |
| CNV | 1.2                              | 2.3                  |

Unprecedented  
classification rates  
<2.5W

| Z7020 | FPS<br>(FPGA) | GOPS/s | BRAM           | LUT              | Latency<br>[us] |
|-------|---------------|--------|----------------|------------------|-----------------|
| LFC   | <b>168k</b>   | 974    | 112<br>(80%)   | 30.6K<br>(57.6%) | <b>102</b>      |
| CNV   | 1.7k          | 190    | 116<br>(82.9%) | 30.0K<br>(56.4%) | <b>2200</b>     |



Ultra-low latency  
for robotics, AR,  
UAVs

## BNN on Pynq

This notebook covers how to use Binary Neural Networks on Pynq. It shows an example of image recognition with a binarized neural inspired by VGG-16, featuring 6 convolutional layers, 3 Max Pool layers and 3 Fully connected layers

### 1. Instantiate a Classifier

Creating a classifier will automatically download the correct bitstream onto the device and load the weights trained on the specified dataset. By default there are three sets of weights

```
In [1]: import bnn
print(bnn.available_params(bnn.NEUTRON_CN))
classifier = bnn.CnnClassifier('cifar10')
['streetview', 'cifar10', 'road-signs']
```

### 2. List the available classes

The CIFAR-10 dataset has 10 classes of images, the names

```
In [2]: print(classifier.bnn.classes)
['Aircraft', 'Automobile', 'Bird', 'Cat', 'Deer']
```

### 3. Open image to be classified

Download a JPEG image of a car and place it in the folder displayed through the notebook

```
In [3]: from PIL import Image
import numpy as np
in = Image.open('/home/xilinx/jupyter_notebooks/')
```

### 4. Launching BNN in hardware

The image is passed into the PL and the inference is required by the network and transferring the image between

```
In [4]: class_out=classifier.classify_image(in)
print('Class number: {0}'.format(class_out))
print('Class name: {0}'.format(classifier.class_))
Inference took 2223.00 microseconds
Classification rate: 449.84 images per second
Class number: 4
Class name: Deer
```

### 5. Launching BNN in software

As a comparison, the same image can be classified using the RUNTIME\_BW parameter to the ImageClassifier

```
In [5]: sw_class = bnn.CnnClassifier('cifar10', bnn.RUNTIME_BW)
class_out = sw_class.classify_image(in)
print('Class number: {0}'.format(class_out))
print('Class name: {0}'.format(classifier.class_))
Inference took 81744.00 microseconds
Classification rate: 1.22 images per second
Class number: 4
Class name: Deer
```

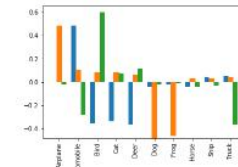
As can be seen, the software implementation is several orders of magnitude slower

### 6. Detailed Classification Info

In addition to highest ranked class, it is possible to get the top 5 results. Take another couple of images of a car and an airplane

```
In [6]: from IPython.display import display
in = Image.open('/home/xilinx/jupyter_notebooks/').in_thumbnail(64, 64), Image.ANTIALIAS
display(in)
car_class = classifier.classify_details(in)
print('Car classes: {0}'.format(car_class))
in = Image.open('/home/xilinx/jupyter_notebooks/').in_thumbnail(64, 64), Image.ANTIALIAS
display(in)
air_class = classifier.classify_details(in)
print('Airplane classes: {0}'.format(air_class))
in = Image.open('/home/xilinx/jupyter_notebooks/').in_thumbnail(64, 64), Image.ANTIALIAS
display(in)
bird_class = classifier.classify_details(in)
print('Bird classes: {0}'.format(bird_class))
Inference took 2195.00 microseconds
Classification rate: 455.58 images per second
Car classes: [258 379 265 169 185 244 249 264 261]
Airplane classes: [378 283 277 277 273 312 312 317 21]
```

```
In [7]: from matplotlib.pyplot import plt
x_pos = np.arange(len(car_class))
fig, ax = plt.subplots()
ax.bar(x_pos, car_class/255, 1, 0.25)
ax.bar(x_pos, (air_class/255), 1, 0.3)
ax.bar(x_pos, (bird_class/255), 1, 0.25)
ax.set_xticks(x_pos)
ax.set_xticklabels(car_class, rotation='vertical')
ax.set_yticks(x_pos)
plt.show()
```



### Classifying multiple images

This example is going to create a string of images from a single input image, filling the image to try and locate an object. This image is a somewhat empty looking highway, and the aim is to find the cars.

```
In [8]: in = Image.open('/home/xilinx/jupyter_notebooks/bnn/motorway.jpg')
in
```

```
In [9]: images = []
bounds = []
for i in range(1, 10):
    stride = i // 2
    x_tilen = in.width // stride
    y_tilen = in.height // stride
    for j in range(x_tilen):
        for k in range(y_tilen):
            bound = (stride * j, stride * k, stride * j + i, stride * k + i)
            if bound[2] < in.width and bound[3] < in.height:
                c = in.crop(bound)
                images.append(c)
                bounds.append(bound)
print(len(images))
247
```

Next the set of images are passed through the classifier. Note how the rate of images is 3 times greater than for single image classification. Once the classes are returned, numpy can quickly find all of the indices of tiles which have matched as cars.

```
In [10]: results = classifier.classify_images(images)
cars = results == 1
indices = cars.nonzero()[0]
```

Inference took 148857.00 microseconds, 570.27 us per image  
Classification rate: 1753.55 images per second

To visualise the output we can draw the bounds of each matched rectangle on the original image.

```
In [11]: from PIL import ImageDraw
draw = ImageDraw.Draw(in)
for i in indices:
    draw.rectangle(bounds[i], outline='red')
in
```



# PYNQ BNN demo

## ➤ LFC

- three-layer fully connected network
  - MNIST(98.4%)
  - Char Recognition

## ➤ CNV

- inspired by BinaryNet and VGG-16
  - Cifar 10(80.1%)
  - SVHN(94.9%)
  - Road Signs

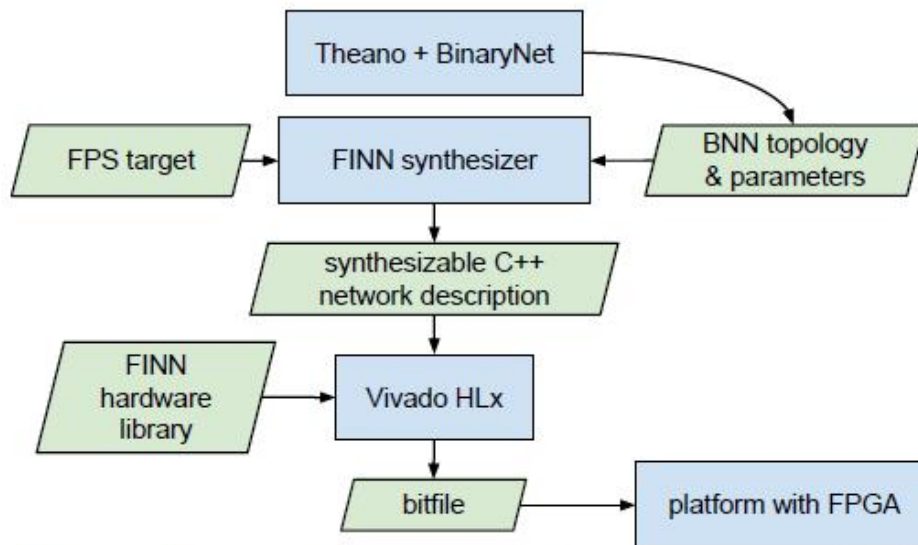


Figure 4: Generating an FPGA accelerator from a trained BNN.

# Summary

- Why use Zynq for DL Inference
- PYNQ-BNN
- Labs:
  - MNIST
  - Char Recognition
  - Cifar 10
  - Road Signs

# Questions?